GENERATIVE AI INTERNSHIP

MONTH-2 PROJECT REPORT

Domain: Generative AI

Tasks Completed

Task 3 :
Implement RAG with Unsloth's Dynamic 4-bit Quantization

Task 4:
Build a Speech-to-Reasoning Pipeline with Whisper + Quantized LLM

Submitted By :

Name: GANDE SANNIHITHA

Internship Domain: Generative AI

Organization: Arch Technologies

Email: sannihitha.gande04@gmail.com

Phone: 9347157778

Internship Month

Month 2

Academic Year

2026

INDEX

S. No.	Contents	Page No.
1	Task 3 – Medical RAG with Unsloth's Dynamic 4-bit Quantization	3
1.1	Objective	3
1.2	Method Followed	3
1.3	Google Colab and T4 GPU Setup	3
1.4	Libraries Used	4
1.5	Dataset and RAG Document Preparation	4
1.6	Document Chunking	5
1.7	Embedding Generation	5
1.8	FAISS Vector Database	6
1.9	Medical Document Retrieval and Testing	6
1.10	Unsloth Dynamic 4-bit LLM and Grounded Answer	7
2	Task 4 – Speech-to-Reasoning Pipeline	8
2.1	Objective and Environment	8
2.2	Libraries Used	8
2.3	Whisper Speech-to-Text	9
2.4	Quantized Reasoning LLM	9
2.5	AI-Generated Voice with gTTS	10
2.6	Multiple Question Testing	10
2.7	Final Workflow and Conclusion	10

TASK 3 – MEDICAL RAG WITH UNSLOTH'S DYNAMIC 4-BIT QUANTIZATION

1.1 Objective

Task 3 was implemented as an extension of the work completed in Task 1 and Task 2.

The main objective was to build a **Retrieval-Augmented Generation (RAG)** workflow for medical question answering.

The workflow takes medical documents, divides them into smaller chunks, converts the chunks into numerical embeddings, stores the embeddings in a vector database, retrieves relevant medical information based on a user's question, and then uses an **Unsloth Dynamic 4-bit Quantized LLM** to generate a grounded answer.

The main objectives were:

- Reuse the medical dataset from the previous task.
- Convert the dataset into RAG documents.
- Divide the documents into smaller chunks.
- Generate embeddings for the medical information.
- Create a FAISS vector database.
- Retrieve relevant medical information.
- Load the Unsloth Dynamic 4-bit model.
- Generate an answer using the retrieved medical context.

1.2 Method Followed

The complete workflow was designed to allow the system to retrieve relevant information before generating an answer.

This approach is useful because the language model does not have to rely only on information stored internally in its parameters. Instead, relevant information can first be retrieved from the medical knowledge base.

1.3 Google Colab and T4 GPU Setup

A new Google Colab notebook was created for Task 3.

The runtime was configured to use a **T4 GPU** before beginning the implementation.

The GPU environment was used to support the processing requirements of the embedding and language-model workflow.

The general setup process was:

1. Open Google Colab.
2. Create a new notebook.
3. Change the runtime to GPU.
4. Select the T4 GPU environment.
5. Install the required libraries.
6. Load the medical dataset.
7. Prepare the RAG pipeline.

LIBRARIES AND DATA PREPARATION

Library	Purpose
`unsloth`	Loading and running the quantized language model efficiently
`sentence-transformers`	Generating embeddings for medical text
`faiss-cpu`	Creating and searching the vector index
`transformers`	Transformer-based language-model functionality
`accelerate`	Supporting efficient model execution

These libraries provided the main components required for the RAG workflow.

2.2 Dataset Used

The medical dataset used in Task 3 was the same dataset used during Task 1.

The dataset contained:**52,002 Medical Q&A records**,the dataset consisted of medical questions and answers and was prepared for use in the Retrieval-Augmented Generation workflow.

The original question-and-answer information was therefore converted into a form that could be searched using embeddings.

2.3 RAG Document Preparation

The first step in implementing the Retrieval-Augmented Generation (RAG) system was to prepare the medical dataset for retrieval. The medical question-and-answer dataset used in the project contained **52,002 medical records**. Each record consisted of medical information in the form of a question, relevant context, and an answer.

The dataset was processed to convert the available medical information into a suitable document format for the RAG pipeline. The question, context, and answer from each record were combined to create meaningful medical documents. This preparation ensured that the information could later be divided into smaller sections and converted into numerical vector representations.

The prepared documents were then used as the input for the subsequent **document chunking and embedding generation** stages. Proper document preparation was important because the quality and structure of the source documents directly affect the accuracy of information retrieved by the RAG system.

The overall preparation process was:

Medical Dataset → Medical Records → Structured Documents → Text Chunking → Embeddings → FAISS Vector Database

This process provided the foundation for building the medical retrieval system and enabled the system to efficiently search for relevant medical information when a user submitted a query.

2.4 Document Chunking

After preparing the medical records, the next step was to divide the documents into smaller and manageable text segments, known as **chunks**. Document chunking was performed to make the medical information easier to process and retrieve efficiently during the RAG process.

The prepared medical documents were split into smaller sections while preserving the important medical information within each chunk. These chunks were then used as the input for the embedding generation stage.

Chunking is an important part of the RAG pipeline because retrieving smaller and relevant portions of a document allows the system to provide more focused context to the language model. This helps the model generate answers based on the most relevant medical information retrieved from the dataset.

The overall process at this stage was:

Prepared Medical Documents → Text Splitting → Smaller Text Chunks → Embedding Generation

2.5 Embedding Generation

After dividing the prepared medical documents into smaller chunks, the next step was to convert these text chunks into **numerical vector representations**, known as embeddings. Embeddings represent the semantic meaning of the medical text in a form that can be processed and compared by the retrieval system.

For this project, **Sentence Transformers** was used to generate embeddings from the medical text chunks. Each chunk was converted into a fixed-length numerical vector. These vectors allow the system to compare the similarity between a user's medical question and the information available in the medical dataset.

The generated embeddings were prepared for storage in the **FAISS vector database**. During the retrieval stage, the embedding of the user's question can be compared with the stored document embeddings to identify the most relevant medical information.

The process followed was:

Text Chunks → Sentence Transformer Model → Numerical Embeddings → FAISS Vector Database

2.6 FAISS Vector Database

After generating the embeddings, the next step was to store them in a **FAISS (Facebook AI Similarity Search) vector database**. FAISS was used to efficiently store and search the numerical vector representations of the medical document chunks.

The generated embeddings were added to the FAISS index, allowing the system to perform similarity-based searches. When a user enters a medical question, the question can be converted into an embedding and compared with the stored document embeddings. The system then identifies the medical chunks that are most relevant to the user's query.

In this project, a **FAISS IndexFlatIP** index was used for similarity search. The vector database therefore acts as the retrieval component of the RAG pipeline, providing relevant medical information to the language model for generating a grounded response.

The process followed was:

Medical Text Chunks → Embeddings → FAISS Index → Similarity Search → Relevant Medical Chunks

2.7 Medical Document Retrieval and Testing

After creating the FAISS vector database, the next step was to test the retrieval process using medical questions. A user query was provided to the system, and the query was converted into an embedding using the same embedding process used for the medical document chunks.

The generated query embedding was compared with the vectors stored in the FAISS index. Based on vector similarity, the system identified and retrieved the medical document chunks that were most relevant to the given question.

For testing the retrieval system, a sample medical question such as **"What are the symptoms of lymphoma?"** was used. The system successfully retrieved relevant medical information from the vector database. This confirmed that the FAISS-based retrieval mechanism was working as expected.

The retrieved medical content was then prepared to be provided as context to the language model in the next stage of the RAG pipeline.

Retrieval process: User Medical Question → Query Embedding → FAISS Similarity Search → Relevant Medical Chunks → Retrieved Context

2.8 Unsloth Dynamic 4-bit LLM and Grounded Answer Generation

After retrieving the relevant medical information from the FAISS vector database, the retrieved content was provided as context to the language model. For this stage, **Unsloth's Dynamic 4-bit quantized LLM** was used to generate the final response.

The retrieved medical information was combined with the user's question and passed to the quantized language model. The model used the retrieved context to generate an answer based on the relevant information available in the medical dataset.

Using the retrieved information as context helps the system provide a **grounded response** instead of generating an answer without reference to the retrieved medical documents. This completes the main RAG pipeline, where information is first retrieved from the medical knowledge base and then used by the language model to generate the final answer.

```
MedInstruct-52k
       ↓
52,002 medical Q&A records
       ↓
75,189 text chunks
       ↓
Sentence Transformer embeddings
       ↓
FAISS Vector Index
       ↓
User Medical Question
       ↓
Top-5 Relevant Chunks
       ↓
RAG Context
       ↓
Unsloth Llama 3.2 1B Instruct
       ↓
4-bit Quantized Inference
       ↓
Grounded Medical Response
```

Task 4 – Speech-to-Reasoning Pipeline with Whisper and Quantized LLM

3.1 Objective and Environment

The objective of Task 4 was to build a **Speech-to-Reasoning pipeline** that can accept spoken questions, convert the speech into text, and use a quantized language model to generate an appropriate response.

The implementation was carried out in **Google Colab** using a **T4 GPU**. The required Python libraries and models were installed and configured in the Colab environment. The pipeline was developed and tested step by step to verify the complete flow from speech input to AI-generated response.

The overall workflow implemented in this task was:

Speech Input → Whisper Speech-to-Text → Transcribed Question → Quantized LLM → Generated Answer

3.2 Libraries and Technologies Used

S. No.	Library / Technology	Purpose
1	**OpenAI Whisper**	Used to convert speech/audio input into text.
2	**Unsloth**	Used to load and run the quantized language model efficiently.
3	**PyTorch**	Used for model execution and GPU-based computation.
4	**SoundFile**	Used for reading and handling audio files.
5	**Librosa**	Used for audio processing and analysis.
6	**gTTS (Google Text-to-Speech)**	Used to generate an AI voice from a text-based question for testing.
7	**Google Colab**	Used as the development and execution environment.
8	**NVIDIA T4 GPU**	Used to accelerate Whisper and quantized LLM processing.

3.3 Whisper Speech-to-Text

The first stage of the Speech-to-Reasoning pipeline was **speech recognition**. For this stage, **OpenAI Whisper** was used to convert the audio input into text.

The Whisper model was loaded in the Google Colab environment and configured to use the available **T4 GPU** for faster processing. When an audio file was provided as input, Whisper processed the speech and generated its corresponding textual transcription.

The transcribed text was then used as the input question for the next stage of the pipeline. This allowed the system to connect speech-based input with the language model.

The process followed was:

Audio Input → Whisper Model → Speech Recognition → Text Transcription → LLM Input

3.4 Quantized Language Model

After converting the speech into text using Whisper, the transcribed question was passed to a **quantized language model** for reasoning and response generation. **Unsloth** was used to load the **Llama 3.2 1B Instruct** model in a 4-bit quantized format.

The 4-bit quantization reduces the memory required to load and run the language model, making it suitable for execution on the available T4 GPU. The model was configured for inference and used the text generated by Whisper as its input.

The language model processed the transcribed question and generated a relevant natural-language response. This completed the reasoning stage of the speech pipeline.

The process followed was:

Whisper Transcription → Text Question → 4-bit Quantized Llama Model → AI-Generated Answer

3.4 AI-Generated Voice and Transcript

To further test the Speech-to-Reasoning pipeline, an AI-generated voice question was created instead of using a personal voice recording. The gTTS (Google Text-to-Speech) library was used to convert a text-based medical question into an audio file.

For testing, the question “What are the common symptoms of anemia?” was converted into an MP3 audio file. This generated audio was then provided as the input to the Whisper model.

Whisper processed the audio and converted the spoken question back into text. The resulting transcript was then passed to the quantized Llama language model for response generation.

This test demonstrated that the complete pipeline could process an AI-generated voice input and successfully convert it into a textual question for the reasoning stage.

The workflow was:

Text Question → gTTS → AI-Generated Voice → Whisper → Transcript → Quantized Llama → Generated Answer

3.5 Speech-to-Reasoning Pipeline Testing

The complete Speech-to-Reasoning pipeline was tested to verify the connection between the speech recognition and language generation stages. The generated AI voice question was provided as the audio input, and Whisper successfully converted the spoken question into its corresponding text transcript.

The transcribed question was then passed to the 4-bit quantized Llama language model through the Unsloth framework. The model processed the question and generated an AI-based response.

The successful execution of these stages confirmed that the implemented pipeline could perform the complete process from audio input to text transcription and finally to AI-generated reasoning output**.**

The final testing flow was:

AI-Generated Audio → Whisper → Text Transcript → Quantized Llama → AI-Generated Response

3.7 Final Workflow and Outcome

The Speech-to-Reasoning pipeline was successfully implemented by integrating speech recognition with a quantized language model. The pipeline accepts an audio question, converts the speech into text using Whisper, and passes the resulting transcript to the quantized Llama model for response generation.

The use of an AI-generated voice through gTTS also provided an additional method for testing the pipeline without requiring a personal voice recording. The successful transcription and response generation demonstrated the working of the complete speech-based reasoning process.

The final workflow of Task 4 was:

```
Audio
   ↓
Whisper
   ↓
 Transcribed Text
   ↓
Quantized Reasoning LLM
   ↓
 Reasoning / Question Answering
   ↓
 Final Answer
```